

Enoncés TDs et TP / Angular v ≥ 17 , partie 1

1. Préparations et installations

Eventuelle installation de nodeJs et npm (si nécessaire):

Si **node -v** et **npm -v** se sont pas des commandes reconnues (dans un terminal CMD ou autre) alors télécharger et installer **nodeJs** (pour windows 64 bits ou autre en version LTS). Relancer ensuite un nouveau terminal et lancer **npm -v** pour vérifier.

Eventuelle installation de typescript (si nécessaire):

Si **tsc -v** est une commande inconnue (dans un terminal), alors lancer la commande suivante :

npm install -g typescript

Installation de angular-cli :

Si **ng -help** est une commande non reconnue , alors lancer la commande suivante :

npm install -g @angular/cli

NB : par défaut , c'est la dernière version stable de @angular/cli qui sera téléchargée.

Si l'on souhaite utiliser une ancienne version, il faut :

- éventuellement désinstaller une autre version (**npm uninstall -g @angular/cli**)

- préciser la version souhaitée : **npm install -g @angular/cli@21.1.0**

Installer si nécessaire l'IDE **Visual Studio code**.

2. TD "création d'un projet Angular"

Dans c:/.../tp-angular ou ailleurs on pourrait éventuellement lancer la commande

ng new --no-standalone my-app-with-module

mais il vaut mieux lancer la commande suivante :

ng new my-app

- l'option **--no-standalone** n'est disponible qu'au sein des versions récentes (17, 18 , 19, ...) et permet d'obtenir la même structure "classique" de l'application créée qu'au sein des versions antérieures d'angular (6,...,15,...)
- choisir plutôt **"css"** ou bien **"tailwind"** comme format de feuilles de styles
- choisir plutôt **"no"** à l'éventuelle question **"enable SSR"** car cette extension peut être ajoutée plus tard (si besoin)

NB: La commande **ng new my-app** met généralement **beaucoup de temps à s'exécuter** et elle **sollicite beaucoup de réseau (nombreux téléchargements)**. Dans certains cas (heureusement très rares) , il faut lancer la commande une seconde fois si la première tentative n'a pas fonctionné.

Se placer dans le répertoire my-app (cd my-app)

Lancer la commande

```
ng serve -o
```

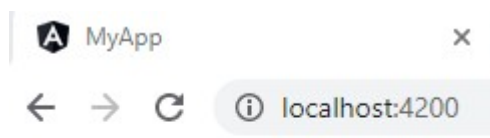
Vérifier le fonctionnement initial de l'application construite via l'url <http://localhost:4200> à charger dans un navigateur .

Via l'éditeur **Visual Studio code** , ouvrir le "folder" **c:...\tp-angular\my-app**

Modifier le fichier **src/app/app[.component].html** en supprimant tout l'ancien contenu et en y plaçant à la place

```
<p> welcome to {{title()}} </p>  <!-- {{title()}} en v19 , {{title()}} en v21 -->
<!-- <router-outlet></router-outlet> -->
```

Vérifier les changements (<http://localhost:4200>)



welcome to my-app

NB: En phase de développement, le mini serveur démarré par **ng serve** régénère automatiquement une nouvelle version à jour de l'application dès que le code .ts ou .html de l'application est modifié. Cependant , certains bugs temporaires ou **certaines modifications importantes de l'application** (configuration, restructuration de composants, ...) **nécessitent quelquefois un redémarrage de ng serve.**

Pour arrêter ng serve avant de le redémarrer, il faut se placer dans le terminal (souvent dédié) où ng serve a été préalablement lancé et taper **Ctrl-C**

Visualiser (au sein de l'éditeur "Visual Studio Code") les fichiers importants suivants (pour comprendre la structure du code source de l'application):

- **my-app/package.json** (repérer les dépendances d'angular en version 17, 18 ou autres)
- **my-app/src/index.html** (repérer la balise `<app-root></app-root>`)
- **my-app/src/app/app[.component].ts** (repérer le sélecteur "app-root")

3. TD "création d'un arbre de composants angular"

Ce TD va montrer comment **créer de nouveaux composants "angular"** et les **rattacher entre eux**. Revenir (si besoin) sur le projet **my-app** (angular) via un *open folder* de *visual-studio-code* et relancer (si besoin) **ng serve** au sein d'un terminal (nouveau ou pas).

NB: depuis la version 21 de `@angular/cli`, la commande **ng g** génère par défaut des fichiers avec des noms simples (ex : `xyz/xyz.ts`). Si l'on souhaite générer des noms de fichiers comportant en plus le type d'élément (ex : `component`, `service`, ...) il faut ajouter la nouvelle option facultative *--type component* ou *--type service* ou *--type ...*

Au sein d'un **nouveau terminal** de *visual-studio-code* lancer (dans l'ordre) les commandes suivantes:

- **ng g component header** *--type component*
- **ng g component footer** *--type component*
- **ng g component basic** *--type component*

se placer dans **src/app/basic** (via `cd`) et lancer les commandes suivantes

- **ng g component calculatrice** *--type component*
- **ng g component tva** *--type component*

Lire les messages affichés par la commande **ng g component ...** et visualiser (au sein de *visual studio code*):

- les nouveaux répertoires créés (dans `src/app`)
- les nouveaux fichiers créés (`.ts`, `.css`, `.html`, `-spec.ts`)

Repérer dans **src/app/header/header[.component].ts** la valeur **app-header** du sélecteur

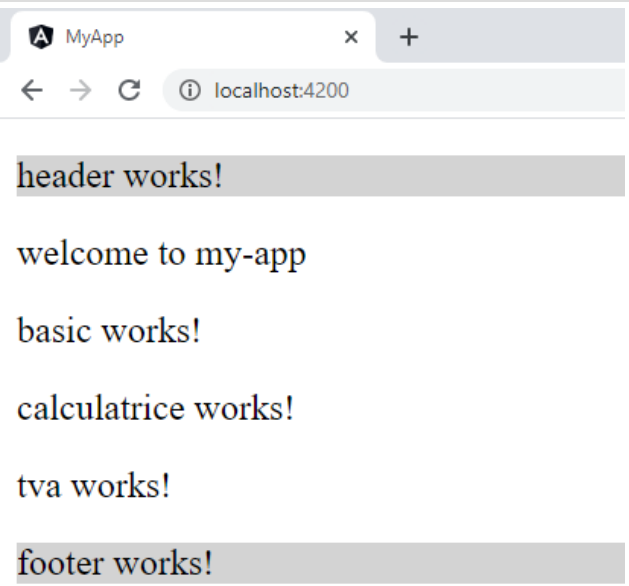
Ajouter les sous composants `<app-header></...>` `<app-basic></...>` `<app-footer></...>` dans **src/app/app[.component].html**

NB : en mode standalone, il faudra prévoir des choses de type `imports: [RouterOutlet, HeaderComponent, FooterComponent]` au sein de `@Component()`

(dans `src/app/app[.component].ts`)

De la même façon **ajouter** les sous-sous-composants *calculatrice* et *tva* dans **src/app/basic/basic[.component].html**

Modifier éventuellement certaines couleurs de fond (via `.css`) pour bien distinguer les sous-composants.

Tester via http://localhost:4200	Etats des principaux fichiers à ce stade
	<pre>src/app/header/header.component.html <p class="entete">header works!</p> src/app/header/header.component.css .entete { background-color: lightgrey;} src/app/basic/basic.component.html <p>basic works!</p> <app-calculatrice></app-calculatrice> <app-tva></app-tva> src/app/app[.component].html <app-header></app-header> <p> welcome to {{title()}} </p> <!-- <router-outlet></router-outlet> --> <app-basic></app-basic> <app-footer></app-footer></pre>

4. TD "calculatrice angular"

Revenir (si besoin) sur le projet **my-app** (angular) via un *open folder* de *visual-studio-code*

Relancer (si besoin) **ng serve** au sein d'un terminal (nouveau ou pas)

Ouvrir le fichier **src/app/basic/calculatrice/calculatrice.component.ts** (créé lors d'un TD antérieur) et y ajouter le code suivant:

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-calculatrice',
  imports: [FormsModule],
  templateUrl: './calculatrice.component.html',
  styleUrls: ['./calculatrice.component.scss']
})
export class CalculatriceComponent {

  a : number = 0;
  b : number = 0;
  res : number = 0;

  montrerHisto : boolean = true;
  historiqueCalculs :string[] = [];

  onCalculer(op:string){
    switch(op){
      case "+" :
        this.res = Number(this.a) + Number(this.b); break;
      case "-" :
        this.res = Number(this.a)- Number(this.b); break;
      case "*" :
        this.res = Number(this.a) * Number(this.b); break;
```

```

        default:
            this.res = 0;
        }
        this.historiqueCalculs.push(`${this.a} ${op} ${this.b} = ${this.res}`)
    }

    //coordonnées relatives de la souris qui survole une div
    x:number=0;
    y:number=0;

    onMouseMove(evt : MouseEvent){
        let currentDiv : HTMLElement = <HTMLElement> evt.target;
        this.x = evt.pageX - currentDiv.offsetLeft;
        this.y = evt.pageY - currentDiv.offsetTop;
    }

    onMouseLeave(evt : MouseEvent){
        this.x=0; this.y=0;
    }

    constructor() { }
}

```

NB: ajouter si besoin **imports: [FormsModule]** dans **@Component({standalone:true, ...})**

Ouvrir le fichier **src/app/basic/calculatrice/calculatrice.html** et y ajouter le code suivant:

```

<div class="c1">
    <h3>calculatrice angular</h3>

    <label>a :</label> <input type="number" [(ngModel)]="a" /> <br/>
    <label>b :</label> <input type="number" [(ngModel)]="b" /> <br/>
    <label>operation :</label>
        <input type="button" value="+" (click)="onCalculer('+')"/>
        &nbsp; <input type="button" value="-" (click)="onCalculer('-')"/>
        &nbsp; <input type="button" value="*" (click)="onCalculer('*')"/>
    <br/>
    <label>resultat:</label>
    <span [style.font-weight]="res>0?'bold':'normal'"
        [class.negatif]="res<0" >
        {{res}}</span> <br/>
    <input type="checkbox" [(ngModel)]="montrerHisto" /> voir l'historique des
    calculs<br/>
    @if(montrerHisto){
        <ul>
            @for(h of historiqueCalculs; track h){
                <li>{{h}}</li>
            }
        </ul>
    }
    <div class="c2" (mousemove)="onMouseMove($event)"
        (mouseout)="onMouseLeave($event)">
        Zone à survoler à la souris .<br/>
        x={{x}} , y={{y}}
    </div>
</div>

```

Ouvrir le fichier **src/app/basic/calculatrice/calculatrice.css** et y ajouter le code suivant:

```

.c1 { background-color: rgb(244, 252, 142)}
.c2 { background-color: rgb(178, 235, 252)}

```

```
li { font-style: italic;}
label { display: inline-block; width: 6em;}
.negatif { color : red; font-style: italic;}
```

`ngModel` est une **directive d'attribut prédéfinie** dans le module `FormsModule`. Ainsi, l'utilisation de `[(ngModel)]` nécessite l'importation de `FormsModule` dans `app.module.ts` ou bien dans `@Component()` en mode standalone :

```
import { FormsModule } from '@angular/forms';
....

@NgModule_ou_@Component({
  imports:      [ ... , FormsModule ],
  ...
})
export class AppModule_ou_XyzComponent { }
```

Attention : sans ajout de `FormsModule` dans la partie `imports: [ ]` la syntaxe `[(ngModel)]="..."` ne fonctionne pas !!! .

Faire fonctionner le code du composant ci-dessus (<http://localhost:4200>) pour bien comprendre son comportement.

calculatrice angular

a :

b :

operation :

resultat: -2

☒ voir l'historique des calculs

- $6 + 8 = 14$
- $6 * 8 = 48$
- $6 - 8 = -2$

Zone à survoler à la souris .

x=204 , y=34

- Analyser chaque ligne de code de l'exemple ci dessus (coté .ts, .html , .css)
- Effectuer éventuellement des ajustements pour voir comment ça réagit
- Le Tp qui va suivre ressemblera partiellement à cet exemple

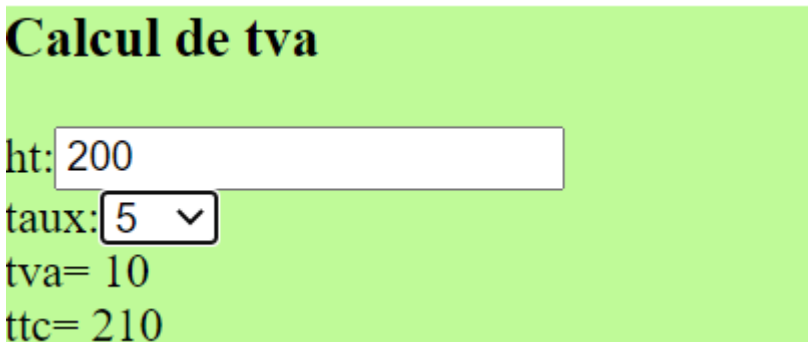
5. Tp "calcul de tva avec listes déroulantes"

Consignes du Tp :

- **coder de A à Z le composant src/app/basic/tva**
- il faudra calculer (via l'appel d'une seule fonction) les montants **tva** et **ttc** à partir d'un montant **ht** saisi et d'un **taux** de tva sélectionné .
- le **taux** de tva (5.0 , 10.0 ou 20.0) sera **choisi** à travers une **liste déroulante**.
- après une première version temporaire où le déclenchement du calcul sera effectué via un bouton poussoir, on codera une version améliorée (sans bouton) où la méthode qui réactualise les valeurs "tva" et "ttc" à calculer sera déclenchée via les traitements d'événements suivants:
 - **(input)="..."** sur **zone de saisie** du montant "ht"
 - **(change)="..."** sur **changement de sélection** du taux .

Rappel: l'événement **input** est déclenché sur une zone de saisie lorsque le texte saisi a changé (après avoir tapé un nouveau caractère).

résultats attendus:



Suite facultative du TP :

Ajouter du côté *tva.component.ts* une map (table d'association) entre taux de tva et liste des principales catégories de produits associés à ce taux.

Initialiser cette map avec quelques valeurs au sein du constructeur.

```
mapTauxCategorieProd= new Map<number,string[]>();
tauxSel : number | undefined = undefined; //taux sélectionné
listeCategoriePourTauxSel = [];
constructor(){
  this.mapTauxCategorieProd.set(20 , [ "services" ,"outils" , "objets"]);
  this.mapTauxCategorieProd.set(10 , [ "transports" ,"hotels" , "restaurants" , "spectacles" , "médicaments"]);
  this.mapTauxCategorieProd.set(5 , [ "aliments" ,"énergies" , "livres" ]);
}
```

Du côté *tva.component.html* , ajouter (en bas) un affichage des différents taux de tva sous forme d'une liste de puces (avec @for() comportant) .

Via un traitement d'événement adéquat, faire ensuite en sorte qu'en cliquant sur un des taux de tva :

- ça actualise (en dessous ou à côté) l'affichage de la liste des catégories de produits associés .
- ce taux de tva sélectionné soit affiché avec une classe de style css différent (pour le mettre en évidence). On pourra pour cela utiliser la syntaxe [class.nomClasseCss]='conditionPourActivation'.

principaux taux de tva: principales catégories pour le taux sélectionné:

- 5% **aliments,énergies,livres**
- 10%
- 20%

6. TP utilisation de "pipe" pour la mise en forme

Au sein de *tva.component.html* , on ajoutera l'utilisation d'au moins un pipe (tel que | **number** : "...") de manière à ce que les résultats des calculs de tva soient toujours affichés avec 2 chiffres au maximum après la virgule.

Après avoir utilisé le pipe prédéfini "number" d'angular (qui a un comportement moyen d'un point de vue "affichage équivoque des grandes valeurs avec séparateur entre millier et centaine ressemblant à une virgule française") , on pourra se programmer un pipe personnalisé "**toFixed**" qui n'affichera pas de séparateur entre milliers et centaines .

Résultats attendus :

Sans pipe	Avec number:'1.0-2'	Avec toFixed:'2'
ht: 123456.7897 tauxTva(%): 20% ▾ tva= 24691.35794 ttc= 148148.14763999998	ht: 123456.7897 tauxTva(%): 20% ▾ tva= 24,691.36 ttc= 148,148.15	ht: 123456.7897 tauxTva(%): 20% ▾ tva= 24691.36 ttc= 148148.15

Indications :

en mode standalone , besoin d'importer DecimalPipe dans @Component .

créer si besoin le sous répertoire src/app/**common/pipe**

ng g pipe toFixed

7. ajustements du composant tva avec signaux

Sauvegarder des copies du composant tva (sans signaux) :

tva.component.html.v1.txt

tva.component.ts.v1.txt

Améliorer le code de tva.component.ts et tva.component.html en s'appuyant sur des signaux.

Indications :

- Les éléments à saisir (ex : ht) ou à choisir (ex : taux) peuvent être déclarés comme des signaux ordinaires (via signal()) .
- Les éléments calculés (ex : tva , ttc) peuvent être matérialisés comme des signaux calculés (via computed())
- la partie tauxPossibles = [5, 10, 20] peut rester simple/ordinaire (sans signal)

8. TD routing/navigation élémentaire

Ce **TD très important** va nous permettre de mettre en place de "routing angular". Grâce à cela, les composants "basic", "welcome" et "login" ne s'afficheront pas tous en même temps mais à tour de rôle selon les choix effectués dans un **menu**.

Revenir (si besoin) sur le projet **my-app** (angular) via un *open folder* de *visual-studio-code* et relancer (si besoin) **ng serve** au sein d'un terminal (nouveau ou pas)

Générer un nouveau composant "welcome" (WelcomeComponent) de premier niveau en lançant la commande suivante au sein d'un terminal (*en étant placé, via cd, dans src/app ou bien à la racine du projet my-app*):

```
ng g component welcome --type component
```

Générer également (si besoin) un composant **"login"** :

```
ng g component login --type component
```

Paramétrer quelques routes simples dans `src/app/app.route.ts` (*anciennement* `src/app/app-routing.module.ts`) (ex: `ngr-welcome/WelcomeComponent`, `ngr-basic/BasicComponent`, `ngr-login/LoginComponent`):

```
import { Routes } from '@angular/router';
import { WelcomeComponent } from '../welcome/welcome.component';
import { LoginComponent } from '../login/login.component';
import { BasicComponent } from '../basic/basic.component';

export const routes: Routes = [
  { path: 'ngr-welcome', component: WelcomeComponent },
  { path: '', redirectTo: '/ngr-welcome', pathMatch: 'full' },
  { path: 'ngr-login', component: LoginComponent },
  { path: 'ngr-basic', component: BasicComponent },
  { path: '**', redirectTo: '/ngr-welcome', pathMatch: 'full' }
];

//NB: les path peuvent éventuellement commencer par "ngr-" .
//ceci permet en production, un éventuel filtrage d'url à un niveau
//intermédiaire (ex: nginx)
```

NB: certaines lignes de type `import { ... } from '...'` peuvent quelquefois être générées par Visual-Studio-Code après un click sur une erreur et un click sur une ampoule jaune ou bleu .

(ré-)intégrer le routage de premier niveau dans `src/app/app[.component].html` : en remplaçant

```
<app-basic></app-basic>
```

par

```
<router-outlet></router-outlet>
```

Ajouter quelques liens hypertextes dans src/app/header/**header.component.html** de type `<a routerLink='/ngr-welcome'> welcome</a>  `;

```
<p class="entete">header
  <a routerLink='/ngr-welcome'> welcome</a> &nbsp;
  <a routerLink='/ngr-basic'> basic</a> &nbsp;
  <a routerLink='/ngr-login'> login</a> &nbsp;
</p>
```

NB : en mode standalone (maintenant par défaut), il faudra prévoir **imports: [RouterOutlet , ...]** dans **app[.component].ts**
et **imports: [RouterLink, ...]** dans **header.component.ts**

Tester le tout (<http://localhost:4200>)